

PES UNIVERSITY
Department of Computer Science and Engineering
Software Engineering
UE24CS341A

Software Architecture and Design Specification

StreamHub – OTT Video Streaming Website

Project: StreamHub – OTT Video Streaming Website (PES University, UE24CS341A Software Engineering)

Version: 1.0

Authors: TEAM-04

Date: 04-10-2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	04-10-2026	TEAM-04	SAD v1.0

Approvals

Role	Name	Signature / Date
Course Coordinator / Project Guide	Arpitha.K Assistant Professor	

1. Introduction

1.1 Purpose

This document specifies the architecture and design of StreamHub, a web-based OTT (Over-The-Top) video streaming platform, as defined in the StreamHub Software Requirements Specification (SRS) v1.0.

1.2 Scope

Covers StreamHub's Viewer-facing services — account/authentication & profile, catalog browse & search, playback, watchlist & ratings, and subscription & payment — together with the Content Administrator's catalog-management interface. StreamHub is delivered as a single responsive website only; native mobile and smart-TV apps are out of scope (SRS §1.2).

1.3 Audience

Developers implementing the React frontend and FastAPI backend, QA engineers, content administrators, and the course coordinator / project guide assessing the deliverable.

1.4 Definitions

OTT (Over-The-Top streaming), VOD (Video on Demand), JWT (JSON Web Token), API (Application Programming Interface), PII (Personally Identifiable Information), RTM (Requirements Traceability Matrix), SPA (Single Page Application).

2. Document Overview

2.1 How to use this document

Provides architectural deliverables including UML diagrams (use-case and sequence), component descriptions, API design, and the security threat model for StreamHub.

2.2 Related Documents

StreamHub SRS v1.0, StreamHub Software Test Plan (STP) v1.0, Requirements Traceability Matrix (RTM — SRS §9).

3. Architecture

3.1 Goals & Constraints

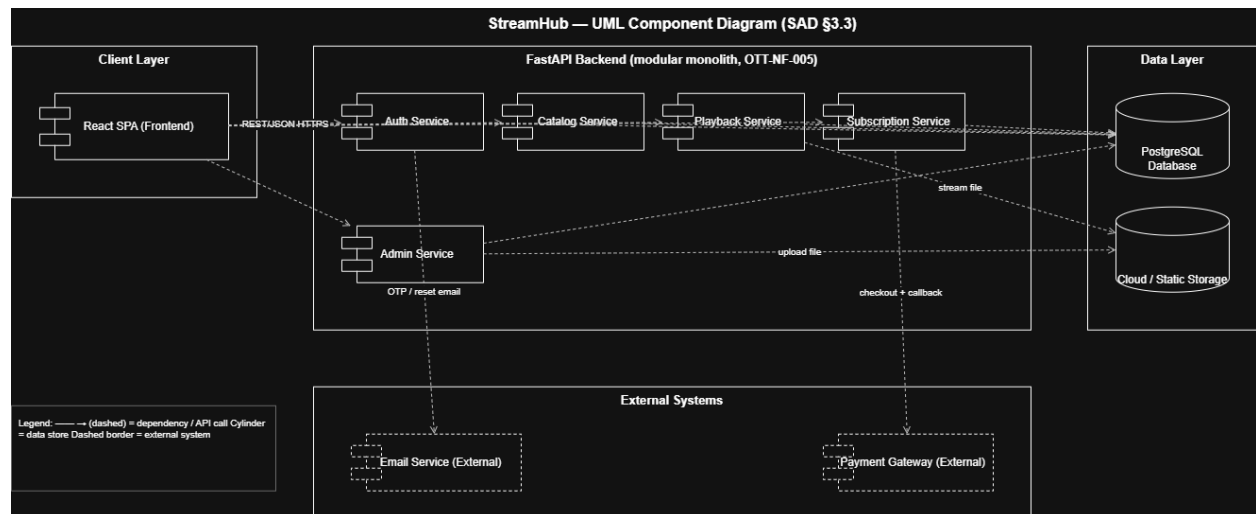
Goals: secure handling of accounts and payments; 95% of API requests responding within 2 seconds (OTT-NF-001); support for at least 50 concurrent simulated users (OTT-NF-004).

Constraints: delivered as a website only, no native mobile or smart-TV app (SRS §2.5); card/payment details never stored on the StreamHub server (SRS §2.5, OTT-SR-004); built and demonstrated within academic project resources, with no dedicated CDN or multi-region infrastructure (SRS §2.5).

3.2 Stakeholders & Concerns

Stakeholder	Concerns
Viewer	Usability, account security, playback reliability
Content Administrator	Ease of uploading and managing the catalog
Course Coordinator / Project Guide	Correctness, completeness, and traceability to the SRS
Developers (TEAM-04)	Modularity and maintainability of the FastAPI backend

3.3 Component (UML) Diagram



3.4 Component Descriptions

- **React SPA (Frontend):** renders the browse/search, title-detail, video player, watchlist, subscription/checkout, and admin dashboard pages.
- **Auth Service (FastAPI):** handles registration, OTP verification, login, JWT issuance, and password reset.
- **Catalog Service (FastAPI):** serves genre-grouped browsing, search, and title-detail data.
- **Playback Service (FastAPI):** streams video, tracks resume position, and records watch history.
- **Subscription Service (FastAPI):** manages plan selection, payment-gateway checkout, and payment-status callbacks.
- **Admin Service (FastAPI):** handles title upload/publish and user/subscription listing for administrators.
- **PostgreSQL Database:** stores accounts, titles, watchlist, ratings, subscriptions, and watch history.
- **Cloud/Static Storage:** hosts video files and poster images.
- **Payment Gateway (external):** hosted checkout for subscription payment and payment-status callback.
- **Email Service (external):** sends OTP verification and password-reset emails.

3.5 Chosen Architecture Pattern and Rationale

A **layered, 3-tier client–server architecture** (React SPA ↔ FastAPI REST API ↔ PostgreSQL) implemented as a modular monolith is chosen, matching the product perspective in SRS §2.1. Microservices are rejected as unnecessary complexity for a single-team academic build at modest scale (50 concurrent users, OTT-NF-004). Instead, the FastAPI backend is organized as clearly separated modules/routers for authentication, catalog, playback, subscriptions, and administration (OTT-NF-005), providing maintainability and separation of concerns without the operational overhead of service discovery, inter-service networking, and distributed transactions.

3.6 Technology Stack & Data Stores

Frontend: React (single-page application). Backend: FastAPI (Python) with Pydantic for input validation (OTT-SR-005). Database: PostgreSQL. Storage: cloud/static storage for video and image files. Transport: HTTPS/TLS 1.2+, REST/JSON (OTT-SR-001). Authentication: short-lived JWT session tokens (OTT-SR-003). Password storage: salted bcrypt hashing (OTT-SR-002).

3.7 Risks & Mitigations

Risk	Mitigation
Payment-gateway callback failure leaves a subscription in an inconsistent state	Idempotent callback handler with a periodic reconciliation check against the gateway
Video storage/bandwidth usage exceeds academic project resources	Serve video as a small number of fixed-quality progressive streams; no adaptive-bitrate transcoding (SRS §2.5)
Single relational database becomes a bottleneck under concurrent load	Connection pooling and indexed queries, sufficient at the stated 50-user scale (OTT-NF-004)

3.8 Traceability to Requirements

Requirement(s)	Component
OTT-F-001, OTT-F-002, OTT-F-003, OTT-F-004 (Register / Login / Reset / Profile)	Auth Module
OTT-F-010, OTT-F-011, OTT-F-012 (Browse / Search / Title Detail)	Catalog Module
OTT-F-020, OTT-F-021, OTT-F-022 (Play / Resume / Watch History)	Playback Module
OTT-F-030, OTT-F-031 (Watchlist / Rating)	Catalog Service / PostgreSQL
OTT-F-040, OTT-F-041, OTT-F-042 (Plans / Payment / Status)	Subscription Module
OTT-F-050, OTT-F-051, OTT-F-052 (Upload / Publish / View Users)	Admin Module

3.9 Security Architecture

Threat Modeling (STRIDE):

- Spoofing → JWT-signed sessions, bcrypt-hashed passwords (OTT-SR-002, OTT-SR-003)
- Tampering → Pydantic input validation on every API route, HTTPS/TLS in transit (OTT-SR-005, OTT-SR-001)
- Repudiation → server-side logging of transaction and payment-status events
- Information Disclosure → TLS 1.2+ enforced, card data never stored on the StreamHub server, no plaintext password logging (OTT-SR-001, OTT-SR-004, OTT-SR-002)
- Denial of Service → rate limiting on authentication endpoints
- Elevation of Privilege → role check on all admin-only routes (OTT-SR-006)

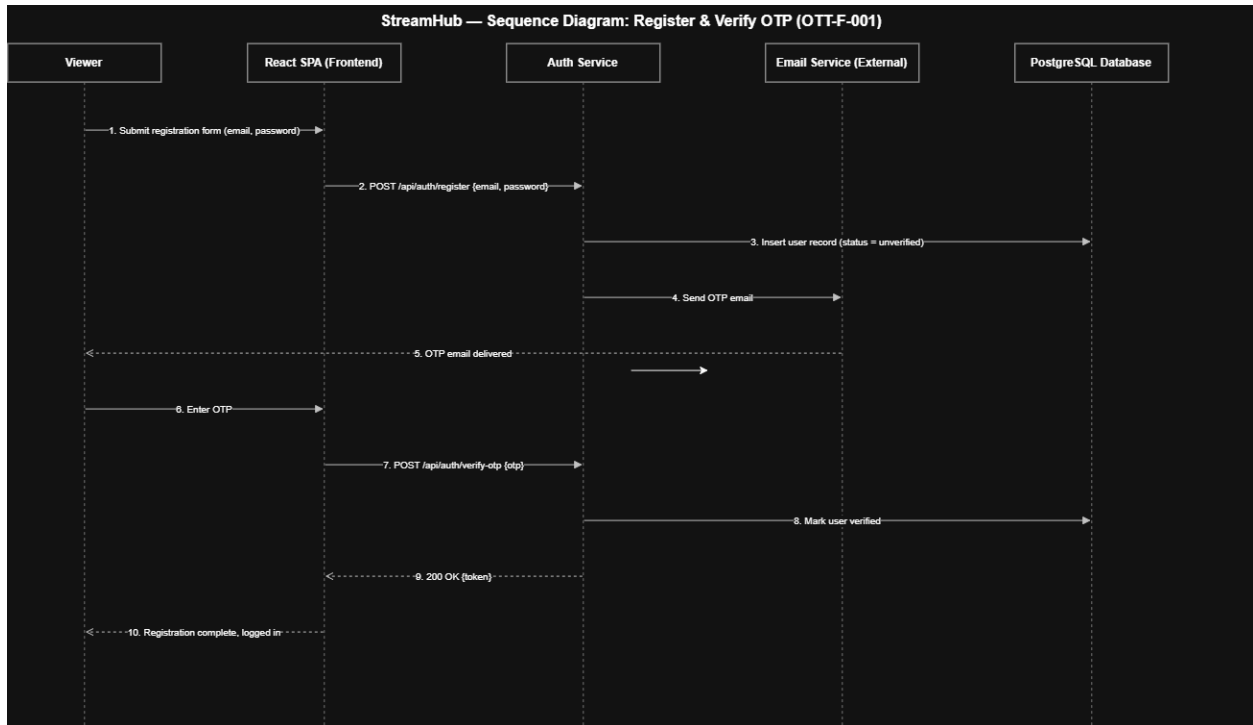
4. Design

4.1 Design Overview

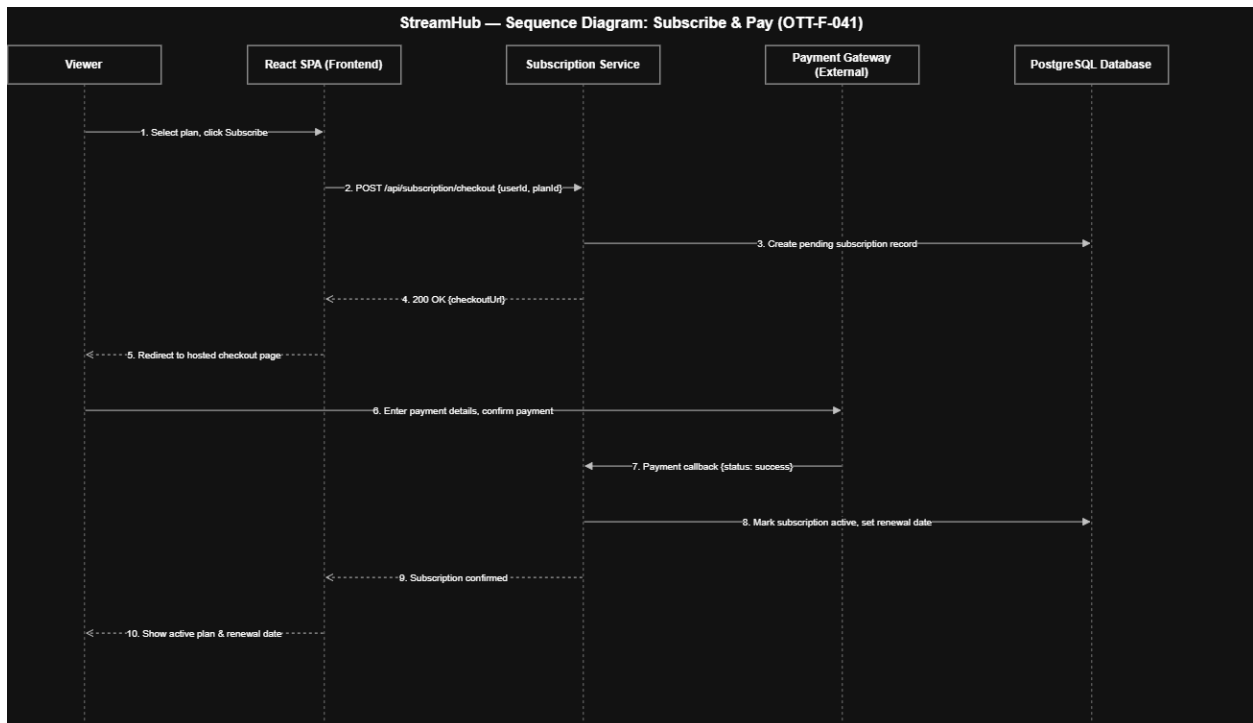
The FastAPI backend is organised into five clearly separated modules — auth, catalog, playback, subscription, and admin (OTT-NF-005) — each exposed as its own router, sharing a single PostgreSQL database. This separation keeps each module independently testable and maintainable without the overhead of a distributed architecture.

4.2 UML Sequence Diagrams

At least two sequence diagrams covering flows specific to StreamHub:



- Register & Verify OTP (Viewer → Auth Service → Email Service → Database)



- Subscribe & Pay (Viewer → Subscription Service → Payment Gateway → callback → Database)

4.3 API Design

Interface definitions for key components:

Endpoint: /api/auth/login

Method: POST

Request: {email, password}

Response: {token, expiresIn}

Errors: 401 Invalid credentials

Endpoint: /api/subscription/checkout

Method: POST

Request: {userId, planId}

Response: {checkoutUrl, status}

Errors: 400 Invalid plan, 502 Payment gateway unavailable

Endpoint: /api/playback/{titleId}/resume

Method: GET

Request: (path) titleId; (auth) JWT

Response: {positionSeconds}

Errors: 401 Invalid or expired session, 404 Title not found

4.4 Error Handling, Logging & Monitoring

Standardised JSON error responses ({status, detail}). No PII or card data is written to logs (OTT-SR-002, OTT-SR-004).

Monitoring covers API response time (OTT-NF-001), video-start latency (OTT-NF-002), concurrent-session count (OTT-NF-004), and uptime during scheduled demo/test windows (OTT-NF-006).

4.5 UX Design

Responsive layout usable on both desktop and tablet widths (OTT-NF-003); accessible contrast and image alt text; a captions field on video metadata (SRS §3.1).

4.6 Open Issues & Next Steps

Future enhancements beyond the current scope: native mobile/smart-TV apps, DRM-protected adaptive-bitrate streaming, and offline downloads — all explicitly excluded from this version (SRS §1.2).

5. Appendices

5.1 Glossary

OTT, VOD, JWT, API, PII, RTM, SPA (see SRS §1.4 for full definitions).

5.2 References

StreamHub SRS v1.0, OWASP guidelines, NIST SP 800-160, IEEE 42010.

5.3 Tools

draw.io (UML diagrams), Swagger/OpenAPI (API documentation), Postman.